IW1RMM CW Encoder — Analisi Routine Software

Versione firmware: v7.1.4 | Hardware: ESP32-2432S028R (CYD) | Autore: IW1RMM

1. Architettura Generale

Il firmware v7.1.4 adotta un'architettura FreeRTOS multi-task con separazione netta tra logica Morse (Core 0) e rendering LVGL (Core 1). La comunicazione inter-task avviene tramite una coda displayQueue e un mutex xMutex.

Task / Contesto	Core	Stack	Priorità	Funzione principale
morseTask	0	8192 byte	2	Keyer paddle, buffer TX, beacon, tune
serialTask	0	4096 byte	1	Parser K3NG e Winkey da Serial
loop() / LVGL	1	Arduino default	—	lv_timer_handler(), processDisplayQueue(), sdPlayerTask()
bleSetupTask	1	8192 byte	1	Init BLE stack (one-shot, si auto-elimina)

Meccanismo	Tipo	Dimensione / Note
displayQueue	QueueHandle_t	64 messaggi DisplayMsg — morseTask→loop()
xMutex	SemaphoreHandle_t	Mutex binario — protegge buffer, wpm, flags condivisi
morseTimer	hw_timer_t 1 kHz	ISR incrementa morseTimerTick ogni 1 ms
morseDelay(ms)	Busy-wait su tick HW	Usata in morseTask al posto di delay()

2. Setup e Inizializzazione

2.1 setup()

Sequenza di inizializzazione eseguita una sola volta su Core 1 (loopTask Arduino).

Step	Operazione	Note v7.1.4
0	Serial.begin(115200)	Serial1.end() preventivo per evitare conflitti

SD	SPI.begin() + SD.begin(SD_CS)	SD inizializzata PRIMA di tft.begin() per evitare reset bus
1	Preferences begin/load	Namespace 'cyd-morse'; carica wpm, freq, vol, ratio, weight, messaggi
2	setupPins()	MORSE_OUT=27 LOW, PADDLE_DAH=22 PULLUP, PADDLE_DIT=35, DAC pin 26, TFT_BL=21 HIGH
3	setupLVGL()	lv_init(), tft.begin(), rotation=1, draw_buf 320×10, registra disp e indev driver
4	setupTouch()	Carica calibrazione da NVS 'touch_cal'; se assente usa hardcoded (220/3730/387/3763)
4b	runTouchCalibration()	Eseguita solo se NVS 'calibrated'==false (primo avvio o dopo reset \J)
5	setupTimer()	timerBegin(1000000), alarm 1ms, ISR onMorseTimer()
6	createUI()	Crea tabview + 5 tab LVGL
7	bleSetup()	Lancia bleSetupTask su Core 1 (one-shot)
7b	setupTasks()	Lancia morseTask (Core 0, pri 2) e serialTask (Core 0, pri 1)

2.2 setupPins()

Pin	GPIO	Direzione	Stato iniziale
MORSE_OUT (PTT/Key)	27	OUTPUT	LOW
PADDLE_DAH	22	INPUT_PULLUP	—
PADDLE_DIT	35	INPUT	—
TFT_BL (Backlight)	21	OUTPUT	HIGH
DAC Audio	26	DAC (DacESP32)	disable()

2.3 setupLVGL()

Inizializza LVGL 8.x con display ILI9341 320×240 e touch XPT2046. Il buffer di disegno occupa 320×10 pixel. La callback my_disp_flush() usa TFT_eSPI::pushColors(); my_touchpad_read() usa XPT2046_Touchscreen su bus HSPI con mappatura coordinate calibrate.

2.4 setupTouch() — NOVITÀ v7.1.4

Carica i parametri di calibrazione touch dal namespace NVS 'touch_cal'. Se non presenti, usa i valori hardcoded di default (x_min=220, x_max=3730, y_min=387, y_max=3763). Inizializza il bus HSPI e la libreria XPT2046_Touchscreen con rotation=1.

2.5 setupTimer()

Configura un hardware timer a 1 MHz con alarm ogni 1000 tick (= 1 ms). La ISR onMorseTimer() (in IRAM) incrementa il contatore morseTimerTick usato da morseDelay() per delay non-bloccanti in morseTask.

3. Calibrazione Touch — NOVITÀ v7.1.4

La v7.1.4 introduce una calibrazione touch interattiva a 4 punti con persistenza NVS in un namespace separato ('touch_cal'), indipendente dalle preferenze principali ('cyd-morse').

Funzione	Firma	Descrizione
touchCalSave()	touchCalSave(int xmin, xmax, ymin, ymax)	Salva 4 valori + flag 'calibrated'=true nel namespace NVS 'touch_cal'
touchCalLoad()	bool touchCalLoad(int &xmin, &xmax, &ymin, &ymax)	Carica valori da NVS. Ritorna true se calibrated==true, false altrimenti
touchCalReset()	void touchCalReset()	Cancella l'intero namespace 'touch_cal' (p.clear()). Usata da onRecalibrate() e comando \J
drawCross()	void drawCross(int x, int y, uint16_t color)	Disegna croce di calibrazione (+15px) + cerchio r=5 sul TFT raw (senza LVGL)
waitTouch()	bool waitTouch(int &rawX, int &rawY)	Attende tocco XPT2046 raw. Timeout 10s → false. Escape via Serial 'R' → false
runTouchCalibration()	void runTouchCalibration()	Procedura completa: 4 punti (TL/TR/BR/BL), validazione range, conferma 10s, save + ESP.restart()

3.1 runTouchCalibration() — Flusso

Fase	Descrizione
Display istruzioni	Schermo nero con testo guida e avviso escape Serial 'R'
4 punti sequenziali	TL(20,20) → TR(300,20) → BR(300,220) → BL(20,220). Per ogni punto: drawCross verde, waitTouch(), conferma rossa
Calcolo valori	xmin=media(TL.x, BL.x), xmax=media(TR.x, BR.x), ymin=media(TL.y, TR.y), ymax=media(BR.y, BL.y)
Validazione range	xmin 100-1000, xmax 3000-4095, ymin 100-1000, ymax 3000-4095, delta>2000 su entrambi gli assi
Conferma	Mostra valori calcolati, attende tocco o timeout 10s (auto-conferma)

Salvataggio	touchCalSave() → ESP.restart() per applicare immediatamente
Fallback	Se annullato/timeout/valori invalidi: ritorna senza salvare, setupTouch() usa hardcoded

4. Preferenze NVS

Funzione	Namespace	Operazione
loadPreferences()	'cyd-morse' (read-only)	Carica wpm, freq, vol, ratio, weight, paddleReversed, beaconIntervalSec, contestCounter, msg0-msg5
savePreferences()	'cyd-morse' (r/w)	Salva gli stessi parametri. Chiamata ad ogni modifica da UI o seriale
loadMemories()	'cyd-morse' (read-only)	Carica k3ngMem[0..25] con chiavi 'mem_A'..'mem_Z'
saveMemory(int idx)	'cyd-morse' (r/w)	Salva una singola memoria K3NG (chiave 'mem_X') — chiamata dopo ogni \P
touchCalSave/Load/Reset()	'touch_cal' separato	Namespace dedicato — p.clear() non tocca 'cyd-morse'

5. Interfaccia Utente LVGL

L'UI è strutturata in un tabview con 5 tab. Tutti gli oggetti LVGL vengono creati su Core 1 in setup() e aggiornati esclusivamente nel loop() tramite la coda displayQueue.

Tab	Funzione createTab*()	Contenuto principale
Morse 1	createTabMorse1()	Controlli WPM/VOL/FREQ, LED TX, KEY mode, pulsanti CQ/NAME/ANT-RIG/CLEAR/QTH/RST/TEST/KEYER, aree log L1/L2/L3
Morse 2	createTabMorse2()	Stessi controlli comuni + RATIO/WEIGHT/TUNE/MACRO/PAD.REV/BEACON/CONTEST/STOP
SD	createTabSD()	Info file, 5 righe testo, SPD-/WPM/SPD+, PLAY/PAUSE/STOP, popup lista file FILE
Settings	createTabSettings()	Mode K3NG/WINKEY, Calibra Touch, Farnsworth, SD LOG, QRSS, Orologio, BLE
Spare	createTabSpare()	Placeholder 'Reserved for future features' (previsto Trainer CW)

5.1 createCommonControls()

Crea i controlli comuni a Tab Morse 1 e Tab Morse 2: bottoni WPM -/+, VOL -/+, FREQ -/+, LED TX circolare (grigio/rosso), label TX, bottone IAMBIC MODE. Accetta puntatori di output per i riferimenti agli oggetti specifici di ogni tab.

5.2 createLogAreas()

Crea le tre aree di testo in fondo al tab (Y=116 e Y=170): due label nere (L1 alta, L2 bassa, font montserrat_12, 316px width, clip, align right) e una label blu (L3 barra TX, font montserrat_16, sfondo 0x0000FF, testo giallo). Aggiornate tramite lvgl_appendTxBar() e lvgl_clearAll().

5.3 Display Queue — processDisplayQueue()

Chiamata ogni ciclo di loop(). Svuota la coda displayQueue ricevendo messaggi DisplayMsg e aggiornando gli oggetti LVGL corrispondenti. Garantisce che tutte le chiamate lv_* avvengano su Core 1.

Tipo messaggio	Payload	Azione LVGL
DISP_TX_LED	u8 (0/1)	lvgl_updateTxLed(): colore LED rosso/grigio su entrambi i tab
DISP_TX_BAR	char	lvgl_appendTxBar(): scroll L3→L2→L1 e append carattere
DISP_CLEAR	—	lvgl_clearAll(): reset stringBuf_L1/L2/L3, label='Ready'
DISP_WPM	i32	lv_label_set_text_fmt su labelWpm1 e labelWpm2
DISP_VOL	i32	Aggiorna VOL label (con 'M' se mute)
DISP_FREQ	i32	Aggiorna FRQ label su entrambi i tab
DISP_IAMBIC_MODE	i32 (KeyerMode)	Aggiorna testo e colore btnIambicMode1/2
DISP_KEYER_LABEL	i32 (0/1)	KEYER (giallo) o BUFFER (verde) su btnKeyerMode
DISP_TUNE	u8	Colore btnTune: rosso=ON, grigio=OFF
DISP_BEACON	u8	Colore btnBeacon: rosso=ON, grigio=OFF
DISP_FARNSWORTH	i32 (wpm o 0)	Label e colore btnFarnsworth: verde=ON, grigio=OFF

5.4 lvgl_appendTxBar(char c)

Implementa lo scorrimento del ticker a 3 righe identico al vecchio sketch VK2IDL originale. L3 (blu, max 32 char) scorre verso L2 (nera, max 42 char) che scorre verso L1 (nera, max 42 char). I nuovi caratteri entrano da destra. Aggiorna tutte e 6 le label (2 tab × 3 righe) ad ogni carattere.

6. Popup e Editor Messaggi

Funzione	Trigger	Descrizione
createPopupBase(title)	Interna	Crea overlay 320×240 semitrasparente + box 240×140 con bordo blu. Ritorna il box come parent
closePopup()	Bottoni SAVE/CANCEL	lv_obj_del(popupBox), azzera popupBox e popupValueLabel
showPopupRatio()	onRatio()	Popup ±0.1 per dahRatio (2.5–4.0). Save aggiorna dahRatio e ricalcola timing
showPopupWeight()	onWeight()	Popup ±0.05 per spacing_Weight (0.80–1.20)
showPopupPadRev()	onPadRev()	Toggle NORMAL/REVERSED con anteprima colore
showPopupBeacon()	onBeacon() short press	Popup ±10s per beaconIntervalSec (10–300s)
showPopupContest()	onContest() 400-1000ms	Popup ±1 per contestCounter (1–999)
showPopupFarnsworth()	onFarnsworthToggle() se OFF	Popup ± WPM effettivo Farnsworth. Mostra 'Effective: N WPM' e 'Chars: M WPM'
showPopupHscw(int wpm)	onWpmUp() se wpm==60 o onWpmDown() se wpm>60	Popup slider 60–100 WPM con snap ogni 5 WPM per modalità HSCW
showMsgEditor(int idx)	Long press (>800ms) su CQ/NAME/ecc	Editor fullscreen 320×240 con textarea + tastiera LVGL uppercase. Save con NVS flush garantito

7. Event Handlers UI

Handler	Evento	Azione
onWpmUp/Down()	CLICKED	Se QRSS attivo: blocca. Se wpm==60 e Up: showPopupHscw(60). Se wpm>60 e Down: showPopupHscw(). Altrimenti ±1 WPM (5–60)
onVolUp/Down()	CLICKED	Scala volumeLevel (0–4) e toneVol (DAC_CW_SCALE). 0=mute
onFreqUp/Down()	CLICKED	±50 Hz audioTone (300–1200 Hz)
onIambicMode()	CLICKED	Cicla IAMBIC_A→IAMBIC_B→MANUAL→AUTO_MODE→IAMBIC_A. Aggiorna testo e colore pulsante direttamente (siamo su Core 1)
handleMsgBtn()	PRESSED/ RELEASED	Short press (<800ms): buffer+=msg+spazio. Long press (≥800ms): showMsgEditor(idx)

onClear()	CLICKED	buffer='', stopRequested=true, isBufferMode=false, postDisplay(DISP_CLEAR)
onTune()	CLICKED	Toggle isTuneMode. Il morseTask gestisce DAC; qui solo colore pulsante
onStop()	CLICKED	stopRequested=true, beaconActive=false, isTuneMode=false, buffer='', reset tutti i flag paddle. digitalWrite(MORSE_OUT,LOW), dac1.disable()
onRecalibrate()	CLICKED	touchCalReset() + delay(500) + ESP.restart()
onProtocolToggle()	CLICKED	Toggle winkeyMode. Se Winkey: Serial.begin(1200). Se K3NG: Serial.begin(115200). Aggiorna label e colore btnProtocol
onFarnsworthToggle()	CLICKED	Se OFF→showPopupFarnsworth(). Se ON→disabilita e ricalcola timing
onSdLogToggle()	CLICKED	Toggle sdLoggingEnabled. Se ON: sdLogInit(). Se OFF: chiude logFile con footer
onQrssToggle()	CLICKED	Cicla QRSS_OFF→3→6→10→30→OFF. Calcola qrssDitMs=mode*1000ms. Blocca WPM se HSCW
onBleToggle()	CLICKED	Se ON→bleStop(). Se OFF e stack pronto→bleInit(). Se stack non pronto→ESP.restart()
onBeacon()	PRESSED/ RELEASED	Se beacon attivo→stop. Se held≥800ms→avvia beacon. Se short→showPopupBeacon()
onContest()	PRESSED/ RELEASED	Long(≥1s)→sendContestMsg()+increment. Med(400-1000ms)→showPopupContest(). Doppio tap rapido→reset a 1

8. Logica Morse

8.1 calculateMorseTiming()

Ricalcola ditLength, dahLength, elemSpace, charSpace, wordSpace_ in base alla modalità attiva. Priorità: QRSS > Farnsworth > Normale/HSCW.

Modalità	ditLength	charSpace	wordSpace_
NORMALE/HSCW	1200/wpm ms	dit×3×weight	dit×7×weight
FARNSWORTH	1200/wpm ms (caratteri)	1200/farnsworthWPM × 3×weight	1200/farnsworthWPM × 7×weight
QRSS_3	3000 ms	9000 ms	21000 ms

QRSS_6	6000 ms	18000 ms	42000 ms
QRSS_10	10000 ms	30000 ms	70000 ms
QRSS_30	30000 ms	90000 ms	210000 ms

8.2 morseToneOn/Off(int ms)

morseToneOn(): se txEnabled=true → digitalWrite(MORSE_OUT,HIGH); se !muted → dac1.outputCW()+enable(); postDisplay(DISP_TX_LED,1); morseDelay(ms); poi spegne. morseToneOff(): solo morseDelay(ms). sendDit/sendDah() chiamano queste due funzioni con la durata appropriata.

8.3 sendFromBuffer()

Preleva il primo carattere dal buffer e lo trasmette. Gestisce i prosign in formato <XX>: cerca '>' con indexOf, estrae il codice e chiama sendProsign(). Per caratteri normali chiama sendMorseChar(). Aggiorna isBufferMode quando il buffer si svuota.

8.4 sendProsign(const String &code)

Invia la sequenza dit/dah del prosign senza inter-element space tra lettere (come da standard ITU). Supporta: AR, SK, BT, KN, AS, SN, HH, CL. Dopo il prosign aggiunge inter-char space. Mostra sul display come <AR> tramite postDisplayChar().

8.5 sendMorseChar(char c)

Trasmette un singolo carattere Morse. Gestisce A-Z, 0-9, e i simboli . , ? / = @ < >. Per spazio: solo morseToneOff(wordSpace-charSpace). Dopo ogni carattere: morseToneOff(charSpace) + postDisplayChar(c).

8.6 postDisplayChar(char c)

Helper che espande i caratteri prosign interni in etichette leggibili prima di inviarli alla coda: '+' → '<AR>', '=' → '<BT>', '~' → '<SK>', '^' → '<SN>', '&' → '<AS>', '(' → '<KN>'. Ogni carattere dell'etichetta viene inviato come messaggio DISP_TX_BAR separato.

9. Tabella mySet[] — Decodifica Paddle

Array di 121 elementi (indici 0–120) per la decodifica albero binario del paddle. morseNum parte da 1; ogni DIT fa morseNum<<=1, ogni DAH fa morseNum=(morseNum<<1)+1. Corrisponde all'ITU-R M.1677 completo con prosign.

Indice	Carattere	Codice Morse	Note

1	' ' (spazio)	root	Word space
2	E	.	
3	T	-	
34	^ (SN)	...-.	SN understood
40	& (AS)	.-...	AS wait
42	+ (AR)	.-.-.	Corretto in v7.1.2 (era 43)
49	= (BT)	-...-	BT paragraph
54	((KN)	-.--.	KN over to you
69	~ (SK)	...-.-	SK end of work
76	?	..-- ..	
85	.	.-.-.-	
90	@	.--.-.	
107	!	-.-.--	
115	,	--..--	
120	:	---...	

10. Paddle Keyer

Funzione	Descrizione
checkPaddle()	Dispatch in base a currentMode: MANUAL, AUTO_MODE, IAMBIC_A, IAMBIC_B. Disabilitata se qrssMode!=QRSS_OFF
paddle_morse_On()	digitalWrite(MORSE_OUT,HIGH) se txEnabled; dac1.enable() se !muted; postDisplay(DISP_TX_LED,1)
paddle_morse_Off()	digitalWrite(MORSE_OUT,LOW) se txEnabled; dac1.disable(); postDisplay(DISP_TX_LED,0)
paddle_beginDIT()	paddle_morse_On(), paddleStartTime=millis(), ditOnFlag=ditToneFlag=true
paddle_beginDAH()	paddle_morse_On(), paddleStartTime=millis(), dahOnFlag=dahToneFlag=true
paddleTimer()	Controlla scadenza DIT/DAH in base a ditLength/dahLength. Al termine: morseNum shift, paddleChar=true
printDitDah()	Se paddleChar e inter-char space ≥ 2×dit: decodifica mySet[morseNum], postDisplayChar(). Se echoEnabled: Serial+BLE

10.1 Modalità MANUAL — Fix v7.1.2

In MANUAL mode, checkPaddle() gestisce press/release del DIT paddle direttamente senza paddleTimer(). Al PRESS: paddle_morse_On() + paddleStartTime=millis() + ditToneFlag=true. Al RELEASE: paddle_morse_Off() + misura durata. Se dur < 2×ditLength → DIT (morseNum<<=1), altrimenti DAH (morseNum=(morseNum<<1)+1). paddleChar=true. La decodifica avviene in printDitDah() come per gli altri modi.

11. morseTask()

Eseguito su Core 0 con stack 8192 byte. Ciclo principale a 2ms con vTaskDelay(2). All'avvio: vTaskDelay(2000ms) per stabilizzazione pin paddle.

Fase nel loop	Condizione	Azione
TUNE	isTuneMode ON (prima volta)	DAC enable, MORSE_OUT HIGH, LED TX ON. Mantiene fino a isTuneMode=false
TUNE	isTuneMode OFF (appena disattivato)	DAC disable, MORSE_OUT LOW, LED TX OFF
PADDLE	buffer vuoto E beacon OFF E tune OFF	checkPaddle() + printDitDah()
BEACON	beaconActive E buffer vuoto	Se elapsed≥beaconInterval: buffer+=cqMessage
BUFFER TX	sempre	sendFromBuffer() (no-op se buffer vuoto)

12. serialTask() e Parser K3NG

Eseguito su Core 0, priorità 1, ciclo 1ms. Chiama checkSerial() che a sua volta chiama checkSerialBle() (BLE) e poi dispatcha su checkSerialWinkey() o parser K3NG in base a winkeyMode.

12.1 Comandi K3NG (checkSerial())

Comando	Parametro	Funzione
\S	nn (WPM)	Set/get WPM. Salva NVS
\F	nnn (Hz)	Set/get frequenza audio (300–1200 Hz)
\L	n (0–4)	Set/get volume. 0=mute
\W	nn (×100)	Set/get spacing weight (80–120 → 0.80–1.20)

\R	nnn (×100)	Set/get dah ratio (250–400 → 2.50–4.00)
\T	—	Toggle tune mode
\A/B	—	Set Iambic A / Iambic B
\K	—	Set Manual key
\O	—	Set Auto mode
\N	0/1	Paddle normal/reversed
\C	—	Clear buffer
\U	—	Query lunghezza buffer
\P	X[testo]	Save/query memoria K3NG A-Z. \P senza param: lista tutte
\M	X	Invia memoria K3NG A-Z
\Q	0/3/6/10/30	Set/get modalità QRSS
\Y	nn	Set/get Farnsworth WPM effettivo (0=disabilita)
\X	—	Toggle TX key output (sidetone only se OFF)
\E	—	Toggle echo paddle su Serial e BLE
\G	±n	Set GMT offset (salva NVS)
\Z	14 cifre	Set datetime GGMMAAAAHHMMSS → settimeofday()
\J	—	Reset calibrazione touch + ESP.restart()
\I/D	—	Stampa status completo
\V	—	Versione firmware
\H	—	Help completo
testo libero	—	Aggiunge caratteri al buffer morse

13. Winkey WK3 Protocol

Attivato da onProtocolToggle() con Serial.begin(1200). Il parser a macchina a stati checkSerialWinkey() gestisce la fase pre-open (solo Admin <00><02>) e post-open (tutti i comandi). Lo status byte viene inviato ogni 500ms o al cambiamento.

13.1 Comandi Winkey implementati (winkeyExecCmd)

Cmd	N param	Funzione
0x00	1	Admin: subcmd 0x02=Host Open (risponde 0x23), 0x03=Host Close, 0x04=Echo,

		0x17→0x31
0x01	1	Sidetone: mappa valore 1-10 a frequenza Hz secondo tabella WK spec
0x02	1	Set WPM
0x03	1	Set Weighting (10-90, 50=normale → spacing_Weight)
0x04	2	PTT Lead/Tail (x10ms) → wkPttLead, wkPttTailTime
0x05	3	Setup Speed Pot (minWPM, range, unused) → wkSpeedPotMin/Range
0x08	0	Backspace: rimuove ultimo char dal buffer
0x09	1	Set PinConfig → wkPinCfg
0x0A	0	Clear Buffer
0x0B	1	Key Immediate: 1=tune ON, 0=tune OFF
0x0C	1	Set HSCW WPM (60-100)
0x0D	1	Set Farnsworth WPM (0=off)
0x0E	1	Set Mode Register (bit 5-4: keyer mode, bit 3: paddle swap)
0x0F	15	Load Defaults: 15 parametri nell'ordine WK3 spec Table 13
0x10	1	Set 1st Extension → wkFirstExtension
0x11	1	Set Key Compensation → wkKeyComp
0x12	1	Set Paddle Switchpoint → wkPaddleSP
0x15	0	Request Status → winkeyUpdateStatus()
0x17	1	Set Dit/Dah Ratio (33-66) → dahRatio
0x1C	1	Buffered Speed Change
0x1D	1	HSCW Speed Change buffered (60-100)
0x20-0x7E	—	ASCII text → buffer morse
0x80-0x85	—	Prosign legacy encoding

13.2 winkeyBuildStatus()

Costruisce lo status byte WK3: tag MSB=110 (base 0xC0). Bit 4=WAIT (buffer>106 char), bit 3=0 (identifier), bit 2=BUSY (buffer non vuoto o key down), bit 1=BREAKIN (paddle attivo), bit 0=XOFF (buffer>106).

14. SD Card

Funzione	Descrizione
sdScanFiles(targetFile)	Scansiona '/' per file .txt (max 20). Se targetFile specificato, legge il contenuto direttamente durante la scansione (evita doppia apertura)
sdLoadFile(filename)	Chiama sdScanFiles(filename) per leggere testo. Normalizza spazi doppi. Aggiorna 5 label testo (50 char/riga). sdState=SD_IDLE
sdPlayerTask()	Chiamata da loop(). Se SD_PLAYING e buffer vuoto: invia UN carattere del testo al buffer morse. isBufferMode=true
sdOpenFilePopup()	Crea popup 300×210 con lista file scrollabile. Ogni bottone carica il file e chiude il popup
sdCloseFilePopup()	lv_obj_del(sdPopup)
sdUpdateFileLabel()	Aggiorna lblSdFile con nome file (max 20 char) e lunghezza testo
sdUpdateInfoLabel()	Aggiorna lblSdInfo con freq e vol correnti
sdUpdateWpmLabel()	Aggiorna lblSdWpm con WPM corrente
sdUpdateButtons()	Aggiorna colori PLAY/PAUSE/STOP in base a sdState
sdLogInit()	Crea/apre /morse_log.txt in append. Scrive header sessione
sdLogWrite(String msg)	Apre /morse_log.txt in append, scrive timestamp [dd/mm/yy HH:MM:SS] + msg, chiude

15. BLE (Bluetooth Low Energy)

Servizio Nordic UART (UUID 6E400001). TX characteristic con NOTIFY, RX characteristic con WRITE. Nome dispositivo 'VK2IDL_Morse'.

Funzione	Descrizione
bleSetup()	Lancia bleSetupTask su Core 1 (stack 8192). Task one-shot: init BLE stack, crea server/service/characteristics, vTaskDelete(NULL)
bleInit()	Configura advertising: nome 'VK2IDL_Morse' nel payload primario (fix v7.1.3), service UUID nello scan response. startAdvertising()
bleStop()	stopAdvertising(), bleEnabled=false, bleClientConnected=false
bleSend(String)	setValue()+notify() se bleEnabled e client connesso
bleSendLong(String)	Invia in chunk da 20 byte con vTaskDelay(10ms) tra i chunk
checkSerialBle()	Processa bleRxBuffer: accumula fino a \n, poi chiama processBleCommand()
processBleCommand(String	Parser comandi BLE: stessi comandi K3NG (\S, \F, \L, \P, \M, \I, \D, \Z, \G, \Y, \Q,

)	\X, \E, \A, \B, \K, \O, \T, \C, \U, \N, \V, \H). Testo libero → buffer morse
updateBleButton()	OFF=grigio, WAIT=blu, ON=verde su btnBle
MyServerCallbacks	onConnect: bleClientConnected=true, pending UI update. onDisconnect: false, restart advertising se ancora enabled
MyCallbacks	onWrite: aggiunge dati ricevuti a bleRxBuffer

16. Orologio RTC Software

Funzione	Descrizione
updateClockLVGL()	Chiamata da loop() ogni 1000ms. Legge time(NULL)+localtime(). Formato: data azzurra + ora gialla con recolor LVGL. Bordo verde smeraldo se anno>1970 (sincronizzato), rosso altrimenti
\Z GGMMAAAAHHMMSS	14 cifre: imposta datetime via settimeofday(). Disponibile da Serial K3NG e BLE
\G ±n	Set gmtOffset (salvato NVS). Usato per visualizzazione locale

17. loop() — Ciclo Principale LVGL

Operazione	Frequenza / Condizione	Note
BLE deferred ops	Ogni ciclo se flag pending	bleUiUpdatePending, bleLogEnabledPending, bleLogDisabledPending
processDisplayQueue()	Ogni ciclo	Svuota coda e aggiorna oggetti LVGL
lv_timer_handler()	Ogni ciclo	LVGL ridisegna e processa touch
sdPlayerTask()	Ogni ciclo	Inietta carattere SD→buffer se SD_PLAYING e buffer vuoto
sdUpdateInfoLabel/WpmLabel()	Ogni 2000ms	Aggiorna info audio e WPM nel tab SD
updateClockLVGL()	Ogni 1000ms	Aggiorna label orologio nel tab Settings
delay(5)	Ogni ciclo	5ms consigliati da LVGL per stabilità

18. Storico Modifiche

Versione	Modifica principale

v6.9.0	Base: keyer morse completo, BLE Nordic UART, SD player, Winkey 2.3, LVGL 5 tab, orologio, QRSS, Farnsworth, beacon, contest
v7.1.1	Refactoring architettura multi-task FreeRTOS. BLE con BLEDevice nativo
v7.1.2	Fix MANUAL mode: checkPaddle() gestisce press/release con misurazione durata DIT/DAH. mySet[] esteso a 121 elementi (ITU completo). Fix indice AR: 43→42. postDisplayChar() riscritta con espansione prosign <XX>
v7.1.3	Fix BLE 'unnamed' su Android: bleInit() con BLEAdvertisementData::setName('VK2IDL_Morse') nel payload advertising primario; service UUID nello scan response
v7.1.4	Calibrazione touch interattiva a 4 punti. Namespace NVS separato 'touch_cal'. Nuove funzioni: touchCalSave/Load/Reset(), runTouchCalibration(), drawCross(), waitTouch(). Comando \J per reset calibrazione via seriale. Protezioni: timeout 10s, escape 'R', validazione range, fallback hardcoded